



UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE FÍSICA DE SÃO CARLOS

INTRODUÇÃO À FÍSICA COMPUTACIONAL - 7600017

RELATÓRIO DE PROJETO REFERENTE A DISCIPLINA
INTRODUÇÃO À FÍSICA COMPUTACIONAL

PROJETO 3: MÉTODOS BÁSICOS INTEGRO-DIFERENCIAIS

Aluno:

Bruno Rossi Carmo N. 11236289

Professor:

Prof. Dr. Francisco C. Alcaraz

São Carlos — 2024

Sumário

1	Questão 1	2
1.1	Introdução	2
1.2	Resultados e códigos	4
1.3	Saída de código	7
2	Questão 2	7
2.1	Introdução	7
2.2	Resultados e códigos	8
2.3	Saída de código	11
3	Questão 3	11
3.1	Introdução	11
3.2	Resultados e códigos	12
3.2.1	Bissecção	12
3.2.2	Newton-Raphson	12
3.2.3	Método da Secante	12
3.2.4	Código e tabelas	13
3.3	Saída de código	18

1 Questão 1

1.1 Introdução

Nesta primeira questão, o objetivo concentrou-se em calcular a derivada de primeira e segunda ordem de uma determinada funções $f(x)$ (Equação 1), utilizando variação dos valores h inseridos no método numérico. A função aqui utilizada foi:

$$f(x) = e^{x^2} \cot(2x) \quad (1)$$

Analisando as derivadas de primeira e segunda ordem, pode-se chegar nas seguintes funções:

$$\frac{df(x)}{dx} = 2e^{x^2} (x \cot(2x) - \csc(2x)^2) \quad (2)$$

$$\frac{d^2 f(x)}{dx^2} = 2e^{x^2} (\cot(2x)(2x^2 + 4\csc(2x)^2 + 1) - x4\csc(2x^2)) \quad (3)$$

Uma vez obtidas as funções, pode-se calcular o resultado da derivada inserindo o valor de x nas funções acima (Equação 2, 3). Porém, o objetivo e desafio desta primeira questão é calcular a derivada de modo computacional a partir de $f(x)$. Para isto, será inicialmente utilizada a definição de derivada tal qual:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Uma aproximação interessante é a chamada **dois pontos para frente** (Equação 4), do qual utiliza-se o ponto de interesse x e um ponto adicional $x + h$ somado com um pequeno h .

$$f'_{2f}(x) = \frac{f(x+h) - f(x)}{h} + \mathcal{O}(h) \quad (4)$$

Vale ressaltar que, ao truncar a série de Taylor, surge um erro de ordem de grandeza representado pelo símbolo $\mathcal{O}(h)$, ou seja, um erro na ordem de grandeza do h admitido. Também pode ser utilizado o conceito para calcular a **derivada para trás de dois pontos**, apresentando mesma ordem de erro para o truncamento (Equação 5).

$$f'_{2t}(x) = \frac{f(x) - f(x-h)}{h} + \mathcal{O}(h) \quad (5)$$

Combinando as duas equações acima, pode-se chegar em uma expressão utilizando três pontos para aproximação da derivada. Esta expressão é chamada de **derivada simétrica de 3 pontos** (Equação 6).

$$f'_{3s}(x) = \frac{f(x+h) - f(x-h)}{2h} + \mathcal{O}(h^2) \quad (6)$$

Notavelmente a expressão apresenta uma vantagem em relação às anteriores, uma vez que seu erro apresenta ordem de grandeza h^2 e, considerando que h seja $0 < h < 1$, então $\mathcal{O}(h^2) < \mathcal{O}(h)$. Indo um pouco além, pode-se também construir via uso de simetrias a **derivada simétrica de 5 pontos** (Equação 7).

$$f'_{5s}(x) = \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h} + \mathcal{O}(h^4) \quad (7)$$

O erro da equação acima apresenta por ser o menor entre as quatro equações apresentadas nesta seção. Uma vez obtida as equações correspondentes às aproximações de derivadas de primeira ordem, pode-se também buscar aproximações numéricas para derivadas de segunda ordem. Deste modo, partindo da seguinte igualdade:

$$f(x+h) - 2f(x) + f(x-h) = f''(x)h^2 + \mathcal{O}(h^4)$$

Aplicando a ideia utilizada anteriormente para derivação das expressões aproximadas, pode-se obter a **derivada segunda simétrica de 3 pontos** (Equação 8).

$$f''_{3s}(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \mathcal{O}(h^2) \quad (8)$$

Seguindo a manipulação algébrica da série de Taylor, pode-se alcançar a equação da **deri-**

vada segunda simétrica de 5 pontos (Equação 9).

$$f''_{5s}(x) = \frac{-f(x-2h) + 16f(x-h) - 30f(x) + 16f(x+h) - f(x+2h)}{12h^2} + \mathcal{O}(h^4) \quad (9)$$

Analisando as duas equações, é possível identificar menor erro sob a equação simétrica de 5 pontos, apresentando uma ordem de erro relativa a h^4 .

1.2 Resultados e códigos

Uma vez dada a função, foi calculada sua derivada de primeira e segunda ordem sob formato analítico. Assim, bastou definir as funções resultado das derivações (Equação 2,3) sobre o código e, em seguida, aplicar as funções sobre o ponto desejado $x_0 = 1/2$. Também foi definida a função inicial (Equação 1) sobre o código para, assim, utilizar os intervalos admitidos para cálculo das derivações numéricas em relações sob o mesmo ponto $x_0 = 1/2$. Deste modo, segue-se o código construído (Listing 1):

```
C  tarefa-1-11236289.f
    implicit real*8 (a-h,o-z)
    parameter (x=1.d0/2.d0)
    open(unit=1,file='saida-1-11236289.dat')
    open(unit=2,file='saida-2-11236289.dat')
C  #Definicoes:#
    ex = exp(x**2) !Exponencial do argumento quadratico.
    cot = tan(2*x)**(-1) !Cotangente.
    csc2 = (sin(2*x)**(-2)) !Cossecante ao quadrado.
C  #Valores reais - derivadas:#
    der1 = 2*ex*x*(cot) - 2*ex*csc2 !Derivada primeira.
    der2 = 2*ex*(cot*((2*(x**2)))+(4*csc2)+1)
    der2 = der2 - 2*ex*4*x*csc2 !Derivada segunda.
C  #Definiremos as funcoes:#
    do i = 1,12
        h = (5.d0**(-i))
        f2t = (fx(x+h) - fx(x))/h
        f2s = (fx(x) - fx(x-h))/h
        f3s = (fx(x+h) - fx(x-h))/(2*h)
        f5s = (fx(x-2*h)-8*fx(x-h)+8*fx(x+h))
```

```

f5s = f5s -fx(x+2*h)
f5s = f5s/(12*h)
f23s = (fx(x+h)-(2*fx(x))+(fx(x-h)))/(h**2)
f25s = -fx(x-2*h) + 16*fx(x-h) - 30*fx(x)
f25s = f25s + 16*fx(x+h) - fx(x+2*h)
f25s = f25s/(12*(h**2))
write(1,*) h,f2t,f2s, f3s,f5s,f23s,f25s
write(2,*) h,abs(f2t-der1),abs(f3s-der1),abs(f5s-der1) !f5s eh
o melhor para iteracao 5.
end do
write(1,'(A,F15.11)') 'Valor exato =', der1
write(1,'(A,F15.11)') 'Valor exato derivada segunda =', der2
C #Fim do programa.#
write(*,*) 'Fim do programa, saidas geradas!'
close(1)
close(2)
end

C #Funcao inicial f(x):#
function fx(x)
implicit real*8 (a-h,o-z)
fx = (dexp((x)**2))/dtan(2*(x))
return
end function

```

Listing 1: Código principal construído no Fortran.

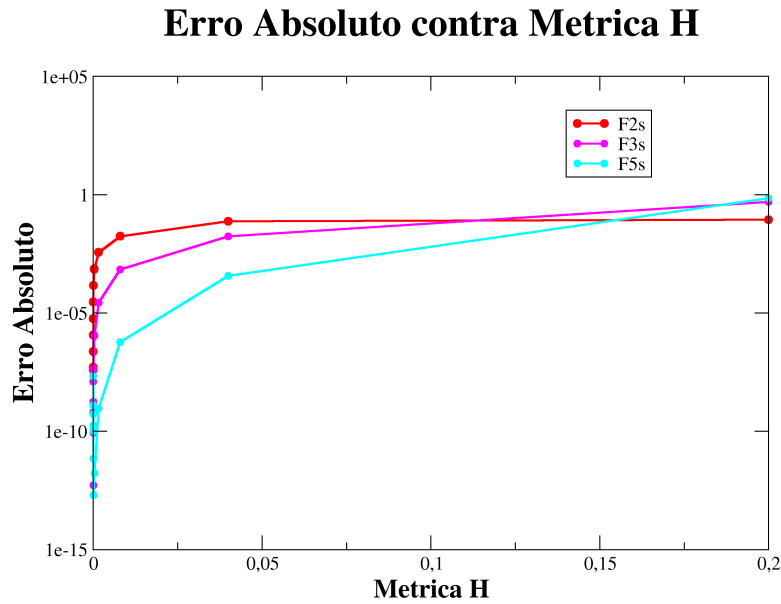
Vale ser ressaltado que, para o código acima, utilizou-se precisão dupla para realização de todos os cálculos. Em relação aos nomes das variáveis atribuídas às operações numéricas, tais variáveis foram nomeadas em relação ao método numérico utilizado.

Em seguida, são apresentados os resultados obtidos para cada modelo em comparação com os valores exatos (Tabela 1). Vale o destaque para a quantidade de casas decimais apresentadas sobre a tabela, do qual se apresentam em torno da ordem de grandeza $\sim 10^{-6}$. Os resultados obtidos no código bruto contém maior quantidade de casas decimais, porém para construção e apresentação dos dados, optou-se pela diminuição para devida representação numérica.

Tabela 1: Resultados obtidos.

h	$f'_{2f}(x)$	$f'_{2t}(x)$	$f'_{3s}(x)$	$f'_{5s}(x)$	$f''_{3s}(x)$	$f''_{5s}(x)$
5^{-1}	-2.714633	-3.874434	-3.294534	-2.098054	5.799005	1.879168
5^{-2}	-2.727936	-2.910990	-2.819463	-2.801977	4.576349	4.533481
5^{-3}	-2.784886	-2.821178	-2.803032	-2.802351	4.536415	4.534762
5^{-4}	-2.798750	-2.806006	-2.802378	-2.802351	4.534830	4.534764
5^{-5}	-2.801627	-2.803078	-2.802352	-2.802351	4.534767	4.534764
5^{-6}	-2.802206	-2.802496	-2.802351	-2.802351	4.534764	4.534764
5^{-7}	-2.802322	-2.802380	-2.802351	-2.802351	4.534764	4.534764
5^{-8}	-2.802345	-2.802357	-2.802351	-2.802351	4.534794	4.534808
5^{-9}	-2.802350	-2.802352	-2.802351	-2.802351	4.536285	4.537238
5^{-10}	-2.802351	-2.802351	-2.802351	-2.802351	4.531626	4.537803
5^{-11}	-2.802351	-2.802351	-2.802351	-2.802351	4.764560	5.007200
5^{-12}	-2.802351	-2.802351	-2.802351	-2.802351	13.234890	22.058150
Exato	-2.80235111004				4.53476386948	

Para buscar o melhor método, foi realizado um gráfico (Figura 5) do erro apresentado para cada modelo individual em função da métrica h utilizada. Posto isso, conclui-se em análise pelo gráfico que a melhor metodologia é a **derivada primeira simétrica de cinco pontos** utilizando as menores métricas de h . Vale ser dito que não foi plotada na imagem a simétrica dois pontos para trás pois seu erro se apresenta na mesma ordem da simétrica dois pontos para frente.

**Figura 1:** Saída visual pós compilação do código descrito.

1.3 Saída de código

Em relação a saída de código, são construídos duas saídas. A primeira representa a tabela descrita anteriormente (Tabela 1). A segunda saída apresenta os valores de h para cada modelo associado ao erro específico contra o valor exato obtido.

```
PS C:\Users\Dobers\Documents\USP\6S\Entregas\proj3> cd "c:\User
9.f -o tarefa-1-11236289 } ; if ($?) { .\tarefa-1-11236289 }
Fim do programa, saídas geradas!
PS C:\Users\Dobers\Documents\USP\6S\Entregas\proj3\tarefa-1>
```

Figura 2: Saída visual pós compilação do código descrito.

2 Questão 2

2.1 Introdução

Nesta questão, objetivou-se calcular via solução numérica a integral definida sob o seguinte formato:

$$I = \int_0^{2\pi} e^{-x} \cos(x) dx \quad (10)$$

Para isto, foram utilizadas três metodologias diferentes para cálculo numérico computacional. Todas utilizam a divisão do intervalo de integração $[0, 2\pi]$ em N valores igualmente espaçados. Este espaçamento se apresenta correspondente a métrica h definida para o método em questão. Por consequência analógica, pode-se definir h do seguinte modo:

$$h = \frac{2\pi - 0}{N} = \frac{2\pi}{N}$$

As metodologias aqui citadas são: **regra do trapézio**, **regra de Simpson** e **regra de Boole**. Todas são, respectivamente, apresentadas a seguir:

$$\frac{1}{2}h(f(x_{i+1}) + f(x_i)) + \mathcal{O}(h^2) \quad (11)$$

$$\frac{1}{3}h(f(x_{i+1}) + 4f(x_i) + f(x_{i-1})) + \mathcal{O}(h^4) \quad (12)$$

$$\frac{2h}{45}(7f(x_{i-2}) + 32f(x_{i-1}) + 12f(x_i) + 32f(x_{i+1}) + 7f(x_{i+2})) + \mathcal{O}(h^7) \quad (13)$$

Vale ser comentado que, nas expressões acima, a anotação x_{i+j} ($j \in \mathbb{Z}$) denota a utilização de pontos acima ou, também, pontos abaixo utilizados ao longo da iteração numérica.

2.2 Resultados e códigos

Para construção deste segundo código, bastou calcular analiticamente o resultado da integração (Equação 14) e, assim, definir a função resultante no código principal construído. Deste modo, aplicando o intervalo em questão, tornou-se possível calcular o resultado exato com ordem de precisão em torno de $\sim 10^{-11}$. Em seguida, utilizou-se a própria função sendo integrada para escrever a função $fun(x)$ no código principal. Uma vez definida a função, foi variado os valores de intervalos N e realizado as iterações computacionais para cálculo aproximado da integração definida sobre os pontos requisitados.

$$\int_0^{2\pi} e^{-x} \cos(x) dx = \frac{e^{-x}(\cos(x) + \sin(x))}{2} \Big|_0^{2\pi} = \frac{1}{2}(1 - e^{-2\pi}) \quad (14)$$

Deste modo, segue o código principal escrito para a questão em análise:

```
C  tarefa-2-11236289.f
    implicit real*8  (a-h,o-z)
    parameter (pi=acos(-1.d0))
    open(unit=1,file='saida-2-1-11236289.dat') !Tabela.
    open(unit=2,file='saida-2-2-11236289.dat') !Erros.
    write(1,' (A15,A31,A28,A29)' ) ' |H|', ' |Trap|', ' |Sims|', ' |Bool|'
C  Integral real:
    real_int = (1.d0/2.d0)*(-dexp(-(2*pi)) + 1) !Integral definida
        calculada.
C  Definindo numero de intervalos:
    do i = 1,12
```

```

i_N = 2**(i+1)
h = (2*pi)/i_N
C   trapezio:
    trp = 0
    err1=0
    do j = 0, i_N
        trp = trp + (h/2.d0)*(fun((j+1)*h)+fun(j*h))
        err1= abs(real_int-trp)
    end do
C   Simpson:
    smp = 0
    err2=0
    do j = 1, i_N, 2
        smp = smp + (h/3.d0)*(fun((j+1)*h)+4*fun(j*h)+fun((j-1)*h))
        err2= abs(real_int-smp)
    end do
C   Boole:
    bool = 0
    err3=0
    do j = 2, i_N, 4
        bool = bool + ((2*h)/45.d0)*(7*fun((j-2)*h))
        bool = bool + ((2*h)/45.d0)*(32*fun((j-1)*h))
        bool = bool + ((2*h)/45.d0)*(12*fun(j*h))
        bool = bool + ((2*h)/45.d0)*(32*fun((j+1)*h))
        bool = bool + ((2*h)/45.d0)*(7*fun((j+2)*h))
        err3= abs(real_int-bool)
    end do
C   Escrever nos documentos:
    write(1,*) '|',h,'|', trp,'|', smp,'|', bool,'|'
    write(2,*) i_N,err1,err2,err3
end do
    write(1,'(A,F15.11)') 'Valor exato =', real_int
    write(*,*) 'Fim do programa, integrais calculadas!'
close(1)
close(2)
end

function fun(x)

```

```

implicit real*8 (a-h,o-z)
fun = dexp(-x)*dcos(x)
return
end function

```

Listing 2: Código principal construído no Fortran.

Uma vez obtido os valores calculados, pode-se construir uma tabela (Tabela 2) apresentando os resultados numéricos obtidos em cada uma das metodologias descritas.

Tabela 2: Resultados obtidos.

N	Regra do Trapézio	Regra de Simpson	Regra de Boole
2^2	0.720451	0.479323	0.453402
2^3	0.552367	0.495535	0.496616
2^4	0.512554	0.498813	0.499031
2^5	0.502608	0.499050	0.499066
2^6	0.500043	0.499065	0.499066
2^7	0.499356	0.499066	0.499066
2^8	0.499162	0.499066	0.499066
2^9	0.499102	0.499066	0.499066
2^{10}	0.499081	0.499066	0.499066
2^{11}	0.499073	0.499066	0.499066
2^{12}	0.499069	0.499066	0.499066
2^{13}	0.499068	0.499066	0.499066
Exato	0.49906627863		

Novamente, para buscar o melhor método, foi realizado um gráfico (Figura 3) do erro apresentado para cada modelo individual em função da métrica h utilizada. Posto isso, conclui-se em análise pelo gráfico que a melhor metodologia é a **Regra de Simpson**.

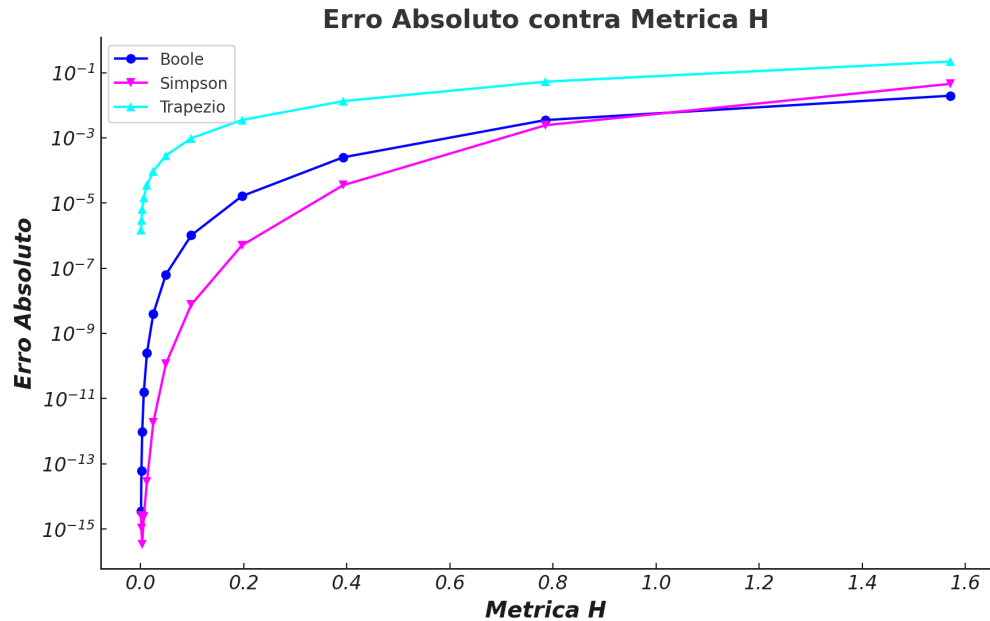


Figura 3: Saída visual pós compilação do código descrito.

2.3 Saída de código

Em relação a saída de código, foram construídas duas saídas. A primeira representa a tabela descrita anteriormente (Tabela 2). A segunda saída apresenta os valores de h para cada modelo associado ao erro específico contra o valor exato obtido.

```
2-11236289.f -o tarefa-2-11236289 } ; if ($?) { .\tarefa-2-11236289 }
Fim do programa, integrais calculadas!
PS C:\Users\Dobers\Documents\USP\6S\Entregas\proj3\tarefa-2>
```

Figura 4: Saída visual pós compilação do código descrito.

3 Questão 3

3.1 Introdução

Para a última questão, serão calculada as raízes reais do polinômio (Equação 15) apresentada a seguir:

$$f(x) = 27x^3 - 522x^2 + 3003x - 4508 \quad (15)$$

Para isto, serão utilizados três principais modelos de busca das raízes: **Método da Bissecção**, **Método de Newton-Raphson** e, também, **Método da Secante**. Como o funcionamento destas metodologias são iterativas, vale explicar cada metodologia implementada sobre a seção dos códigos já descritos.

3.2 Resultados e códigos

3.2.1 Bissecção

O método da Bissecção consiste em, dado um intervalo $[b, a]$ onde $b < a$, buscar mudanças de sinal ao longo do intervalo sobre a função. Para isso, a iteração segue o seguinte modelo:

- **1** : Definir um valor médio $x_m = \frac{a+b}{2}$.
- **2** : Analisar o sinal, de modo que caso $f(x_m)f(a) > 0$ implica na próxima iteração definir $x_m = a$. Caso contrário, define-se na próxima iteração $x_m = b$.
- **3** : Por fim, quando o módulo absoluto $|a - b|$ da iteração ser menor do que 10^{-6} , então é encerrada as iterações.

O código será apresentado ao final deste tópico já apresentando as três metodologias. Deste modo, antes de realizar os métodos, foi realizada uma busca direta sobre a função para encontrar as raízes. Uma vez encontrada, os valores são armazenados e aplicados nos intervalos $[b, a]$ da metodologia.

3.2.2 Newton-Raphson

O método baseia-se no uso da derivada primeira do polinômio em análise e, também, de um termo inicial x_0 . Assim, para cada iteração é realizada a definição de um novo x_n de modo que: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$. O critério de parada encontra-se quando o módulo absoluto de $x_{n+1} - x_n$ for menor do que a tolerância ϵ . Assim, como descrito anteriormente, o código também será apresentado no fim da seção.

3.2.3 Método da Secante

O método da secante apenas ajusta Newton-Raphson caso a derivada não esteja disponível. Assim, a iteração se torna: $x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$. Note que este método agora utiliza dois pontos para as iterações. Assim, como descrito anteriormente, o código também será apresentado no fim da seção.

3.2.4 Código e tabelas

O código construído se configurou do seguinte modo:

```
C      tarefa-3-11236289.f
      implicit real*8 (a-h,o-z)
      dimension termo(5)
      open(unit=1,file='saida-1-11236289.dat')
      write(1,' (A10,A31,A28,A25)' ) ' |N|', ' |Raiz|', ' |b|', ' |a|', ' |Erro|'
      ,
      open(unit=2,file='saida-2-11236289.dat')
      write(2,' (A10,A31,A28,A25)' ) ' |N|', ' |Raiz|', ' |Erro|'
      open(unit=3,file='saida-3-11236289.dat')
      write(3,' (A10,A31,A28,A25)' ) ' |N|', ' |Raiz|', ' |b|', ' |a|', ' |Erro|'
      ,
C      Busca direta:
      j=1
      termo(j) = -10.d0 !Vetor com valores de intervalo esperado.
      do i = -100,100
          val=fun(real(i,8)*0.1d0)*fun(real(i+1,8)*0.1d0)
          if (val.lt.0.d0) then
              j = j + 1
              termo(j) = real(i,8)*0.1d0
          end if
      end do
C      Realizar metodos:
      do i = 1,3
C          #Bisseccao:#
          err = 1
          N = 0
          dir = termo(i+1)+0.1d0
          esq = termo(i)
          do while (err.gt.10.d0**(-6))
              x_med = (dir+esq)/2.d0
              if(fun(dir)*fun(x_med).gt.0.d0) then
                  dir = x_med
              else
                  esq = x_med
              end if
          end while
      end do
```

C

```

        N = N + 1
        write(1,*) N, x_med,esq,dir,err
        err = abs(dir-esq)
    end do
    write(1,*) '-----'
    #Newton:#
    err = 1
    N = 0
    x0 = termo(i+1)
    do while (err.gt.10.d0**(-6))
        x0 = x0 - (fun(x0)/dev(x0))
        err = abs(fun(x0)/dev(x0))
        N = N + 1
        write(2,*) N, x0, err
    end do
    write(2,*) '-----'
    #Secante:#
    err = 1
    x0 = termo(i+1)
    x01= termo(i)
    N = 0
    do while (err.gt.10.d0**(-6))
        xef = x0
        x0 = x0 - fun(x0)*((x0-x01)/(fun(x0)-fun(x01)))
        err = abs(fun(x0)*((x0-x01)/(fun(x0)-fun(x01))))
        x01 = xef
        N = N + 1
        write(3,*) N,x0,x01,x0,err
    end do
    write(3,*) '-----'
    #Fim#

```

C

```

end do
close(1)
close(2)
close(3)
end

function fun(x)

```

```

implicit real*8 (a-h,o-z)
fun = 27.d0*(x**3.d0)-522.d0*(x**2.d0)+3003.d0*(x)
fun = fun-4508.d0
return
end function

function dev(x)
implicit real*8 (a-h,o-z)
dev = 81.d0*(x**2.d0)-1044.d0*(x)+3003.d0
return
end function

```

Listing 3: Código principal construído no Fortran.

Em geral, o código utilizou três iterações gerais para, em cada iteração, realizar as iterações em buscas das raízes utilizando os métodos especificados. Tal organização se deu em relação ao vetor $termo(i)$ responsável por ter os termos identificados pela busca direta correspondentes (ou próximos) às raízes reais.

Como iniciou-se a busca direta entre o intervalo $[-10, 10]$, foram localizadas três regiões próximas a possíveis raízes. Deste modo, serão apresentados os resultados para três diferentes tabelas.

Vale ser dito que todas as iterações iniciaram dos mesmos pontos, sendo possível comparar seus resultados e, também, demora para alcançar a raiz desejada. Deste modo, analisando o resultado obtido para a primeira raiz real (Tabela 3) é possível notar que os métodos da secante e bissecção utilizam muito mais iterações do que Newton-Raphson. O mesmo é observado para as outras duas raízes apresentadas (Tabela 4,5). Deste modo, conhecendo por busca direta os arredores das raízes somado com o método Newton-Raphson apresenta forte otimização de processo de procura das raízes reais.

Tabela 3: Resultados obtidos pelos métodos Bisseccção, Newton-Raphson e Secante.

Iteração	Bisseccção		Newton-Raphson	Método da Secante	
	$x_{i,-}$	$x_{i,+}$		x_{i-1}	x_i
1	-3.800000	2.400000	2.332972	5.872135	2.300000
2	-0.700000	2.400000	2.333333	2.493403	2.303675
3	0.850000	2.400000		2.303675	2.333012
4	1.625000	2.400000		2.333012	2.333302
5	2.012500	2.400000		2.333302	2.333333
6	2.206250	2.400000			
7	2.303125	2.400000			
8	2.351563	2.351563			
9	2.327344	2.351563			
10	2.327344	2.339453			
11	2.327344	2.333984			
12	2.327344	2.333984			
13	2.331848	2.333984			
14	2.333164	2.333984			
15	2.333092	2.333984			
16	2.333204	2.333398			
17	2.333374	2.333398			
18	2.333327	2.333398			
19	2.333327	2.333398			
20	2.333340	2.333398			
21	2.333334	2.333398			
22	2.333334	2.333398			
23	2.333334	2.333398			
24	2.333334	2.333398			
Exato			2.33333333330		

Tabela 4: Resultados obtidos pelos métodos Bisseccção, Newton-Raphson e Secante.

Iteração	Bisseccção		Newton-Raphson	Método da Secante	
	$x_{i,-}$	$x_{i,+}$		x_{i-1}	x_i
1	5.000000	7.700000	7.664990	7.600000	5.872135
2	6.350000	7.700000	7.666666	5.872135	7.649204
3	7.025000	7.700000	7.666667	7.649204	7.661934
4	7.362500	7.700000		7.661934	7.666633
5	7.531250	7.700000		7.666633	7.666667
6	7.615625	7.700000			
7	7.657813	7.700000			
8	7.657813	7.678906			
9	7.657813	7.668359			
10	7.663086	7.668359			
11	7.665723	7.668359			
12	7.665723	7.667041			
13	7.666382	7.667041			
14	7.666382	7.666714			
15	7.666547	7.666714			
16	7.666629	7.666714			
17	7.666629	7.666670			
18	7.666650	7.666670			
19	7.666650	7.666668			
20	7.666660	7.666668			
21	7.666660	7.666668			
22	7.666663	7.666668			
23	7.666663	7.666668			
Exato	7.66666666670				

Tabela 5: Resultados obtidos pelos métodos Bissecção, Newton-Raphson e Secante.

Iteração	Bissecção		Newton-Raphson	Método da Secante	
	$x_{i,-}$	$x_{i,+}$		x_{i-1}	x_i
1	8.500000	9.400000	9.334195	9.300000	8.647291
2	8.950000	9.400000	9.333334	8.647291	9.363998
3	9.175000	9.400000		9.363998	9.307152
4	9.287500	9.400000		9.307152	9.332739
5	9.287500	9.343750		9.332739	9.333345
6	9.315625	9.343750		9.333345	9.333333
7	9.329688	9.343750			
8	9.329688	9.336719			
9	9.333203	9.336719			
10	9.333203	9.334609			
11	9.333203	9.334426			
12	9.333203	9.334283			
13	9.333313	9.334283			
14	9.333313	9.333467			
15	9.333327	9.333467			
16	9.333327	9.333405			
17	9.333332	9.333405			
18	9.333332	9.333386			
19	9.333319	9.333386			
20	9.333327	9.333386			
Exato			9.33333333330		

Assim, conclui-se que tanto o método da secante quanto o método Newton-Raphson apresentam grande vantagem operacional em relação ao método da bissecção.

3.3 Saída de código

Em relação a saída de código, foram construídas três saídas. Uma saída para cada metodologia aplicada ao encontro das raízes. A saída de tela do código apenas apresenta que o programa foi devidamente executado.

```
PS C:\Users\Dobers> cd "c:\Users\Dobers
Fim do programa, raízes encontradas!
PS C:\Users\Dobers\Documents>
```

Figura 5: Saída visual pós compilação do código descrito.